© 2026 IEEE. This is the author's version of the article that has been published in the proceedings of IEEE Visualization conference. The final version of this record is available at: xx.xxxx/TVCG.201x.xxxxxxx/

Topological Equivariant Artist Model

Hannah Aizenman*
CUNY Graduate Center

Mikael Vejdemo-Johansson†
CUNY College of Staten Island
CUNY Graduate Center

Thomas Caswell‡
National Synchrotron Light Source II
Brookhaven National Lab

Michael Grossberg§
City College of New York
CUNY Graduate Center

Abstract

The contract data visualization tools make with their users is that a chart is a faithful and accurate visual representation of the numbers it is made from. Motivated by wanting to make better tools, we propose a methodology for fully specifying arbitrary data to visualization mappings in a manner that easily translates to code and provides a consistent interface for accessibility tools to query for information about the state of the visualization. We propose that fiber bundles provide a uniform interface for describing a variety of underlying data - tables, images, networks, etc. - in a manner that independently encodes the mathematical structure of the topology and the fields of the dataset. Modeling the data structures that store the datasets as sheaves provide a method for specifying visualization methods that are designed to work regardless of how the dataset is stored - whether the data is on disk, distributed, or on demand. Specifying the visualization library components as natural transforms of sheaves means that the constraints that the component must satisfy to be structure preserving can be specified as the set of morphisms on the data and graphic sheaves, including the structure on the topology and fields of the data. Using category theory to formally express how visual elements are constructed means we can translate those expectations into code, which can then be used to enforce the expectation that a visualization tool is faithfully translating between numbers and charts. The sheaf formalism is generic enough that with appropriate choices of a sheaf structure to replace the graphic sheaves, the same framework will work to model and specify non-visual modalities with identical underlying data representations, and structurally similar data to perception transformations.

1 Intro

Motivated by wanting to make better tools, we propose a methodology for fully specifying arbitrary data to visualization mappings in a manner that easily translates to code. Furthermore, this methodology provides a consistent interface for accessibility tools to query for information about the state of the visualization. Generally, preserving structure means that a visualization is expected to preserve the `field` properties and `topology` of the corresponding dataset, where **field** is a set of values of the same type and **topology** is the connectivity and relative positioning of elements in a dataset [**?**]. Field structure is traditionally codified as the Steven's measurement scales [**?**], where each scale is a set of actions on a group. Topological structure is generally assumed by visual algorithms [**?**, **?**], but generally these algorithms do not verify that input structure.

For example, a `line` algorithm often does not have a way to query whether a list of (x,y) coordinates is the distinct rows, the time series, or the list of stations in **??**. We propose that the bar plot, line plot, and heatmap can be verified as structure preserving because they have a homeomorphic relationship to the 0D (•) points, 1D (–) linear, and 2D (■) surface continuities embedded in the continuous 3 dimensional. Our methodology generalizes Bertin[**?**]'s codification of structure, Mackinlay's[**?**] mathematical formalization, Wilkinson's set theoretic approach [**?**], and Kindlemann and Scheidegger's algebraic visualization design (AVD) framework [**?**] by explicitly incorporating topology, supporting non-group and non-monoidal structures, and providing a framework for translating the theoretical ideas into buildable components.

Throughout this paper we will be discussing visualization primarily, but by changing the last of the three fiber bundles (the graphic bundle) we discuss in Section **??**, we could go from a visual representation to an auditory, haptic, or any other modality and retain the benefits we identify from the fiber bundle and sheaf structures.

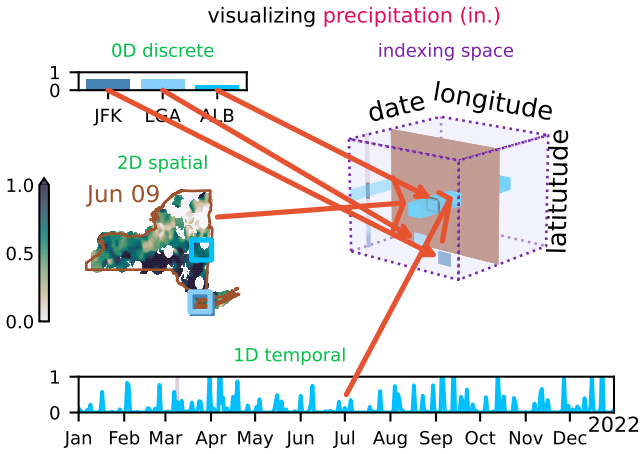


Figure 1: This weather station data has multiple embedded continuities - points at each time and position, timeseries at each position, and maps at each time - that are expected to be homeomorphic to the continuity of their respective visualizations.

2 Data Model

Table 1: Summary of how the data is abstracted using topological structures

	point/openset/base index/subset/indices	fiber record/fields	total data type	Fiber bundle
Data	$k \in \mathcal{U} \subseteq K$	$r \in F$	E	$F \hookrightarrow E \to K$
Visual	$k \in \mathcal{U} \subseteq K$	$V \in P$	V	$P \hookrightarrow V \to K$
Graphic	$s \in W \subseteq S$	$d \in D$	H	$D \hookrightarrow H \to S$

*e-mail: haizenman@gradcenter.cuny.edu
†e-mail: mvj@math.csi.cuny.edu
‡e-mail: tcaswell@bnl.gov
§e-mail: mgrossberg@ccny.cuny.edu

© 2026 IEEE. This is the author's version of the article that has been published in the proceedings of IEEE Visualization conference. The final version of this record is available at: xx.xxxx/TVCG.201x.xxxxxxx/

Table 2: Functions that associate topological subspaces with records

	section record at location	sheaf set of possible records for subset
Data	$\Gamma(K, E) \ni \tau : K \to F$	$\mathcal{O}_{K,E} : U \to \Gamma(U, E\restriction_U)$
Visual	$\Gamma(K, V) \ni \mu : K \to P$	$\mathcal{O}_{K,V} : U \to \Gamma(U, V\restriction_U)$
Graphic	$\Gamma(S, H) \ni \rho : S \to D$	$\mathcal{O}_{S,H} : W \to \Gamma(U, H\restriction_W)$

We model data as a fiber bundle because it can encode topological properties, field types, and data values in a uniform manner. We extend Butler's proposal of using fiber bundles as an abstraction for visualization data [**?**, **?**] by incorporating Spivak's fiber bundle representation of relational databases [**?**, **?**]. As we show in Table **??** and Table **??**, where Spivak represents a database schema with the bundle's fiber space F topological indexing through a base space K and a specific dataset as a section of the bundle τ and we introduce parallel analogous structures representing visual and graphic specifications.

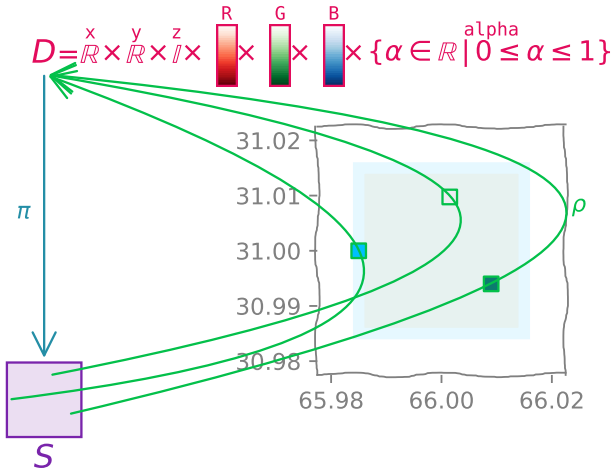


Figure 2: The scatter marker is specified by the section ρ, which maps from a point $s \in S$ into the fiber D to retrieve the values that compose into the pixel (approximated as a square). The section evaluated over the entire space $\rho|_S$ returns the entire scatter mark, shown here in faded form to make it easier to see the individual pixels.

By choosing a topological space $\mathcal{T}$ as indexing space underlying the dataset we get a uniform abstraction for describing arbitrary continuity – time series, lists of observations, spatial fields all have the same abstract structure – and we get a method for managing overlapping embedded topologies, such as we show in **??**. Spivak's fiber bundle construction of types decouples field types from field names and single-dimensional and multi-dimensional fields share a common abstraction: they all rely on some choice of mathematical object for the data types. A particular dataset can in this paradigm be encoded as a τ of a bundle, assigning to each index value an observation in the data types prescribed by the fiber space. This fits topology and field types into a data definition `dataset : topology → field`. We abstract these data containers as sheaves $\mathcal{O}$, as summarized in **??** and the gluing property of sheaves provides a systematic way to track continuity of the data [**?**] and apply visual algorithms developed on this model regardless whether the data fits in memory, is distributed, or streaming.

3 Artist

We call the mapping from data to graphic $A : \tau \mapsto \rho$ an Artist. Artists are named for the artist class in Matplotlib because the Artist class (and its children) assemble and manage the visual elements produced in Matplotlib[**?**]. We propose that Artists (the transform and the library implementation) are structure preserving when they are implemented as sheaf maps from the data representation sheaf to the graphics sheaf. This means that the artist satisfies the following constraints, as illustrated in **??**:

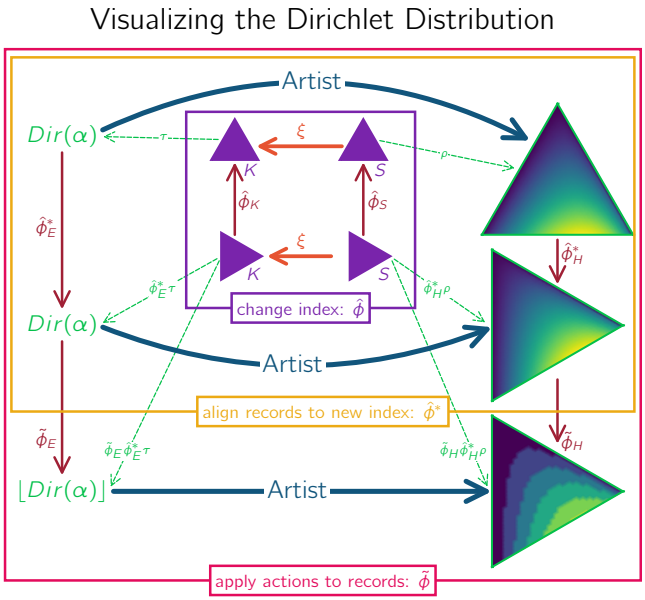


Figure 3: The transposition of the Dirichlet distribution is preserved in the visualization because the Artist implements presheaf constraints and the binning of the Dirichlet data is preserved in the visualization because the artist is equivariant.

homotopic there exists a deformation retract[1] $\xi : S \to K$ between graphic and artist base space

topology the artist is commutative w.r.t. the presheaf morphism $\hat{\phi} : k' \mapsto k$ and pullback $\hat{\phi}^*\tau\restriction_{U'} : \tau\restriction_{U'} \mapsto \tau\restriction_{U' \circ \hat{\phi}}$: index changes on the record leave values in the record unmodified $\tau(K) = \hat{\phi}^*\tau(k') = \tau(\hat{\phi}(k'))$

fields the artist is commutative w.r.t. actions on the fiber. An action $\tilde{\phi} : F \to F$ induces a function between sections $\tilde{\phi} : \hat{\phi}^*\tau\restriction_{U'} \mapsto \hat{\phi}^*\tau'\restriction_U$. We also require $\tilde{\phi}$ to respect the structure, such as operators, of F.

With this underlying topological structure to describe the visualization pipeline, traditional constructions from topology and category theory generate natural operators on the data structures.

Pushouts and pullbacks, of central importance in category theory and topology, emerge here as operators to add and extend records, respectively. Since (in categorical language) sheaf maps preserve limits and colimits, this observation shows that artists are composable: $A(\bigsqcup_i K_i, \prod_j F_j) = \bigsqcup_i \prod_j A(K_i, F_j)$

[1] A deformation retract is required because it ensures a continuous mapping between graphic and data continuity when a screen is 2-dimensional and the data continuity is ¡2D]

© 2026 IEEE. This is the author's version of the article that has been published in the proceedings of IEEE Visualization conference. The final version of this record is available at: xx.xxxx/TVCG.201x.xxxxxxx/

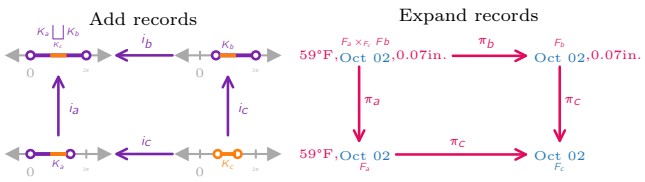


Figure 4: Pullback (left) and pushout (right) structure of the operations that add records (increasing the indexing base space) or expand records (merging different fiber spaces with a database join-like operation).

ν_i	μ_i	$\mathbf{codomain}(\nu_i) \subset P_i$
position	x, y, z, theta, r	$\mathbb{R}$
size	linewidth, markersize	$\mathbb{R}^+$
shape	markerstyle	$\{f_0, \ldots, f_n\}$
color	color, facecolor, markerfacecolor, edgecolor	$\mathbb{R}^4$
texture	hatch	$\mathbb{N}^{10}$
	linestyle	$(\mathbb{R}, \mathbb{R}^{+\,n, n\%2=0})$

Table 3: Some of the P components of the V bundles in Matplotlib components

As illustrated in **??**, this deformation retract ξ gives rise to two so-called transport functors (ξ_*, ξ^*) – known as the pushforward ξ_* and pullback[2] ξ^*. These functors also codify how visual elements correspond to distinct data elements, which is a necessary condition of a visualization being readable[**?**]. The pushforward ξ_* matches each point in the data space to the specification of the graphic at that point $\xi_*\rho(k) = \rho\restriction_{\xi^{-1}(k)}$, while the pullback ξ^* matches each point in the graphic space to the data over that point $\xi^*\tau(s) = \tau(\xi(s)) = \tau(k)$. If the map ξ was explicitly implemented with the corresponding sheaves, the information in $\tau, \xi^*\tau, \rho, \xi_*\rho$, as illustrated in **??**, could be exposed to an accessibility tool (like a screen reader). This functionality could, for example, compensate for a major accessibility limitation of the current Matplotlib architecture, which is that the interactive widgets and GUIs are rendered as PNGs (so that Matplotlib does not have to maintain a GUI library) which means that screen readers cannot query the individual elements.

4 Construction

Artists naturally factorize into a two-stage process, starting with an encoding stage ν followed by a compositing stage Q. In the encoding stage ν, a data bundle is mapped to a visual variable bundle ν. Since E and $\nu\mathtt{total}$ are structurally identical, any V can be redefined as E; therefore, as shown in **??**, any collection of ν functions can be composed such that they are equivalent to a ν that directly converts the input to the output.

$$F_k \xrightarrow{\nu} P_k := F'_k \xrightarrow{\nu'} P'_k \qquad \nu'' \tag{1}$$

As with artists, ν are maps of sections such that operators can also act on transformers ν, meaning that encoders can be added $\nu_{a+b} = \nu_a + \nu_b$ and multiplied d $\nu_{a\times b} = \nu_a \nu_b$.

The fibers of ν are an internal library specification of graphics. For example, **??** shows a standardized internal representation of the visual parameters used by Matplotlib functions. One of the ways Matplotlib facilitates accessibility is by allowing the user to customize most of the aesthetics - colors, fonts, textures. Standardizing the internal parameters provides for a consistent interface for external accessibility tools, such as libraries of accessible stylesheets or tools that may want to query the Matplotlib Artist for context of the current state.

The compositing stage Q then assembles the visual components into a visual element. Since encoder functions are infinitely composable, as described in **??**, a new compositor function Q can be constructed by pre=composing ν functions with the existing Q.

$$\Gamma(K, V) \xrightarrow{\nu} \Gamma(K, V') \xrightarrow{Q} \Gamma(S, H) \qquad Q' \tag{2}$$

The composition in **??** means that different measurable components can yield the same visual elements. Operators can also act on compositors Q such that $Q_{a+b} = Q_a + Q_b$ and multiplied d $Q_{a\times b} = Q_a Q_b$.

As illustrated in **??**, the factorization of the artist into the encoding and composition stages provides a framework for testing each independently. For example, encoders build in this framework would be designed to ensure that a transformation from a non colorblind safe colormap into a colorblind safe colormap would ensure that the colormaps are equivalent such that the transformation is not introducing a change in structure. Similarly, the composition stage can be tested to ensure that parameters are applied consistently across visual elements and, as with encoding, more accessible elements can be swapped in without introducing new structure.

5 Conclusion

We have demonstrated a mathematical formalism for representing the architecture of components in a data visualization library, in which the mathematical structures both enforce well known principles of data visualization and suggest factorizations and decompositions the result in highly modularizable infrastructure, such that artists can be constructed from composable primitive. This decomposability leads directly to a test framework ambition: if we can split $A = Q \circ \nu$, and each factor ν, Q can be further decomposed into composable primitives, we can focus testing on each of these building blocks reducing testing for correctness of the process as a whole to correctness of each building block, and correctness of their chosen combinations. The composability of this framework allows for staged adaptation such that it can be incorporated into an existing library, as exemplified by `https://github.com/matplotlib/data-prototype`. Furthermore, while the graphics sheaves here deal with visual properties – pixel positions and colors, et.c. – one could easily imagine picking a different choice for the fiber D in the realization sheaf that tracks information about haptic or auditory expressions, resulting in an identical abstraction, with many overlapping building blocks, for multimodal data representations.

Supplemental Materials

This paper reuses figures from a longer version of this paper; therefore all code for the figures can be found in `https://github.com/story645/team/paper/figcode`. Additionally, there is an attached supplement with a glossary of math terms used in this paper.

[2] different use of the word pullback than in the preceding paragraph – an unfortunate collision of terminology

© 2026 IEEE. This is the author's version of the article that has been published in the proceedings of IEEE Visualization conference. The final version of this record is available at: xx.xxxx/TVCG.201x.xxxxxxx/

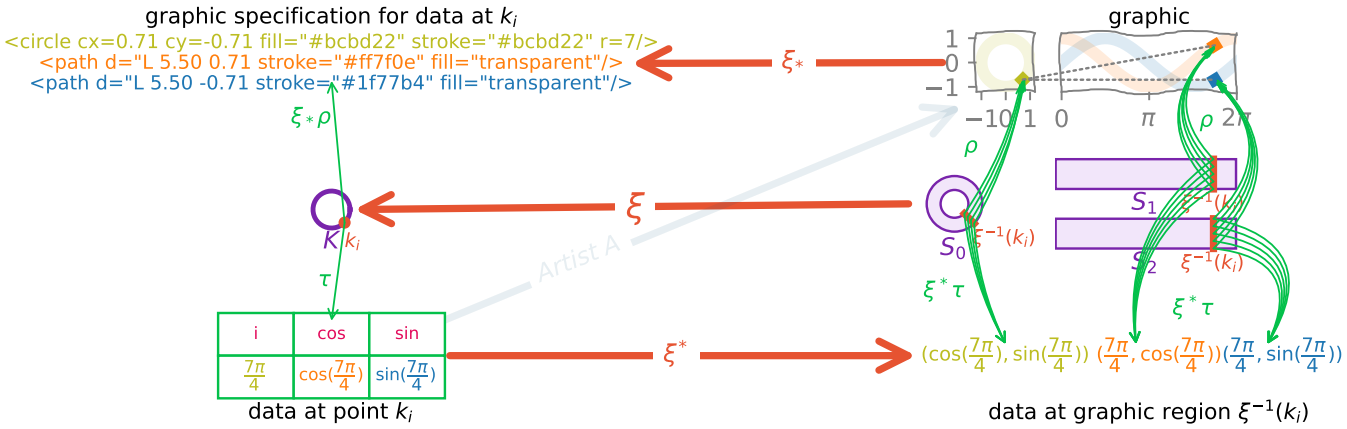


Figure 5: A deformation retract ξ that relates the graphics footprint S_0 of an abstract circle to the circle itself gives rise to a pair of additional maps. The pushforward ξ_* matches a point in the graphical realization to the specification of the graphic at that point, while the pullback ξ^* matches each point in the graphic space to the data giving rise to that point.

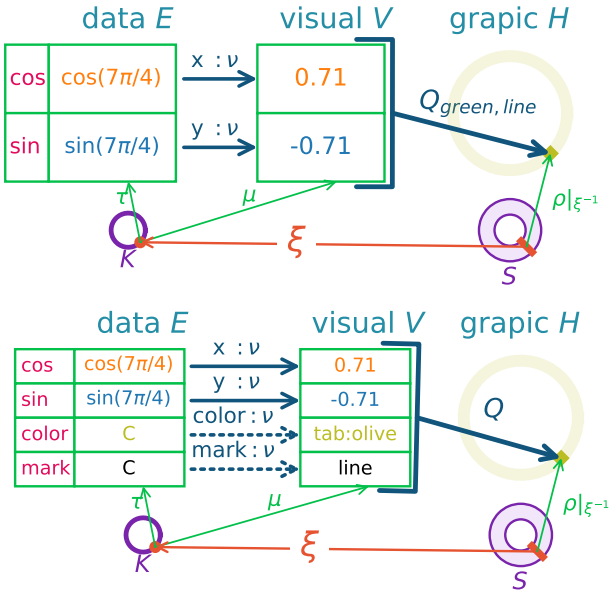


Figure 6: The compositor Q can be fully specified $\mathrm{Q}_{\mathtt{green,line}}$ such that the only input is the encoding of the data values. Alternatively, the compositor Q supports more customizability by encoding a constant parameters in the visual bundle, which pulls back to constant parameters in the data bundle.

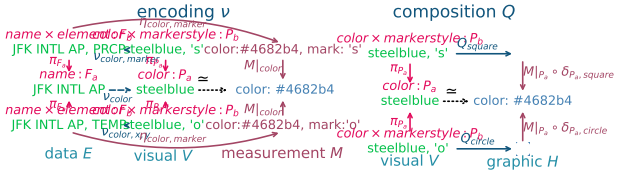


Figure 7: The composability of the model allows for separation of concern in testing: the encoding functions $\mathtt{vchannel}$ are tested to ensure that they convert the data into the internal specification, while the compositors are tested to ensure that the visual specifications are encoded consistently.

Initiative DAF, an advised fund of Silicon Valley Community Foundation

Acknowledgements

The authors would like to thank the anonymous reviewers who gave constructive feedback on an earlier version of this paper. The authors are also grateful to the various Matplotlib and Napari contributors, particularly Juan Nunez-Iglesias, and Nicolas Kruchten for their valuable feedback from the library developer perspective. Hannah is also very grateful to Nicolas for the suggestion of augmented notation and to the nlab and wikipedia contributors who wrote clear explanations of many of the topics discussed in this paper. This project has been made possible in part by grant number 2019-207333 (EOSS Cycle 1) and EOSS Cycle 3 Chan Zuckerberg